

# **Persistência, Exceções e Refatoração no Dendê Eventos**

**Autor:** André Almeida Gomes Neto  
Augusto Brandão Ferreira  
Deivid Souza dos Santos Oliveira  
Hellen Verena da Conceição Magalhães  
Ismael Rodrigues de Oliveira Neto  
João Vítor da Conceição de Almeida

# Agenda

Evolução da API Dendê Eventos com JDBC e MySQL, focando em persistência, organização, tratamento de exceções customizadas e escalabilidade.

## **1. Introdução:**

Evolução da Persistência:  
HashMap para Banco  
Relacional MySQL

## **2. Diagrama de Sequência:**

Fluxo do User Story  
Visualizar Perfil do Usuário  
Visualizar Perfil do Organizador

## **3. Persistência com JDBC**

Integração com Banco de dados, ConfigProperties e ConnectionPool.

## **4. CrudRepository e repositories:**

Padronização da persistência.

## **5. Construção dos Repositories:**

Implementação de Operações CRUD.

## **6. RowMapper:**

Mapeamento entre Banco e Objetos.

## **7. Tratamento de exceções:**

Exceções customizadas e RuntimeException.

# Introdução

**Problema**

**Solução**



**De: HashMap (Memória)**

**Para: MySQL (Persistência)**

Implementação de CrudRepository, RowMapper, ConnectionPool, ConfigProperties, exceções customizadas e diagrama de sequência para modularização e padronização da persistência.

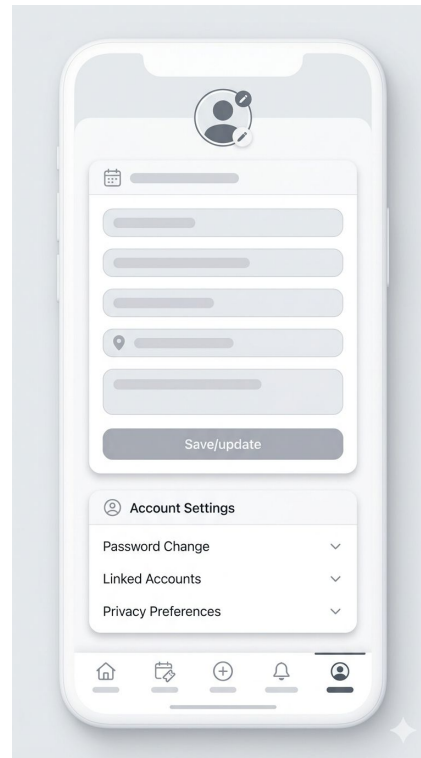
Substituição do repositório em memória por persistência real no Mysql usando JDBC.

# Diagrama de Sequência

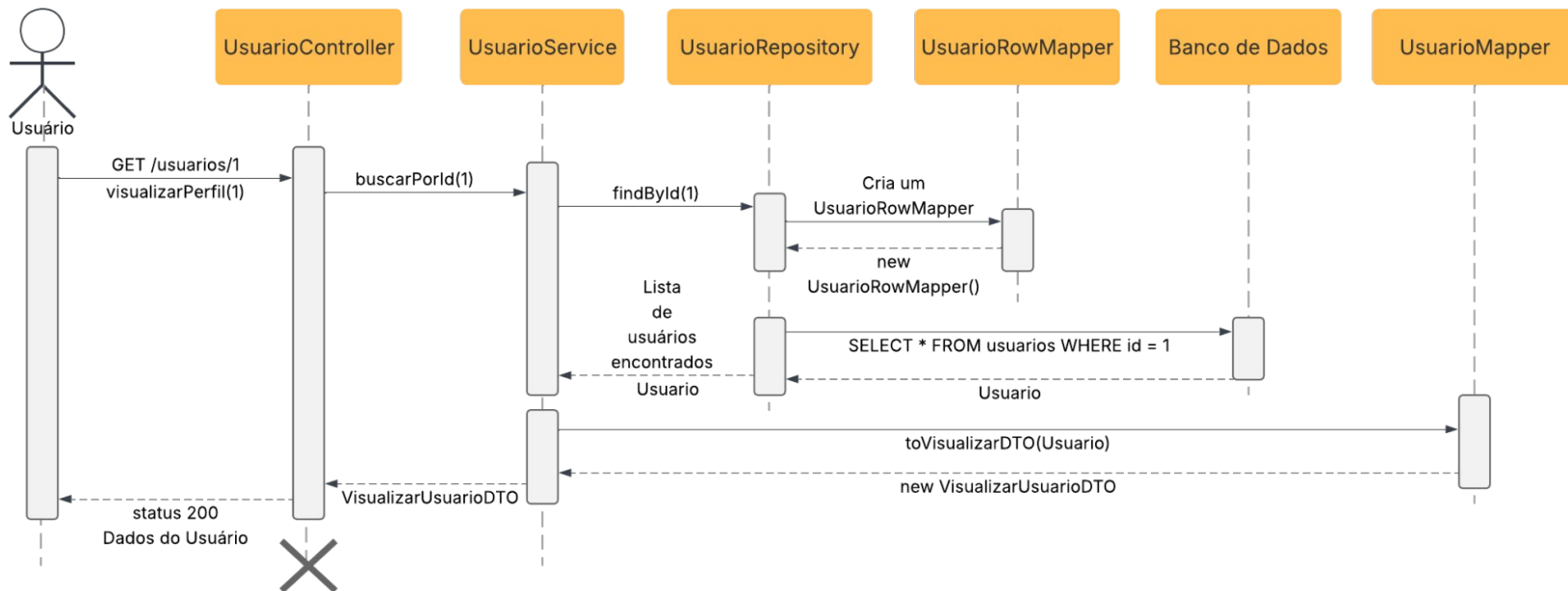
## Caso de Uso:

### Visualizar Perfil do Usuário:

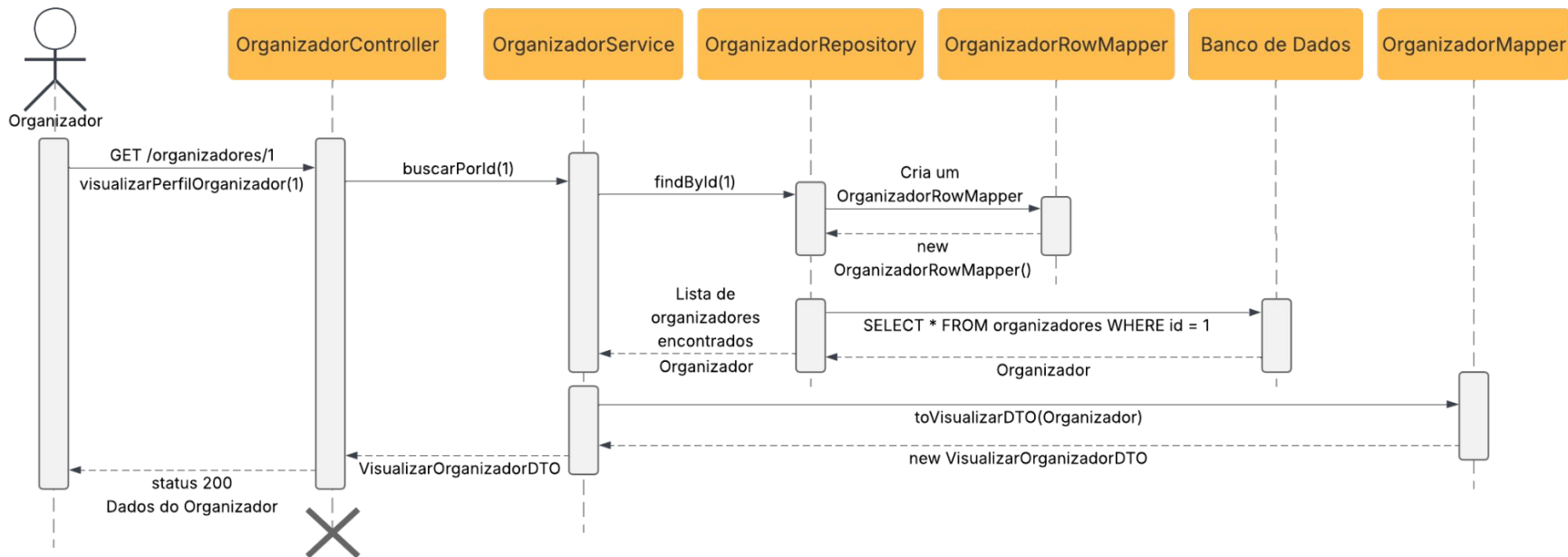
- Visualizar perfil do usuário comum.
- Visualizar perfil do organizador.



# Diagrama de Sequência

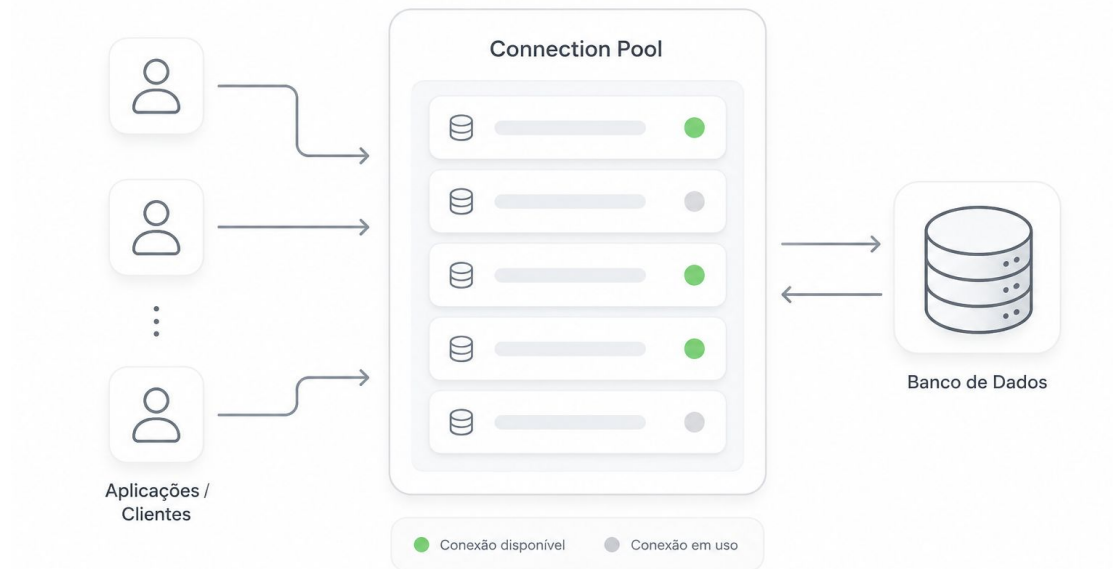


# Diagrama de Sequência



# Persistência com JDBC

## Connection Pool



**HikariCP: Eficiência no reuso de conexões.**

# Persistência com JDBC

## ConfigProperties

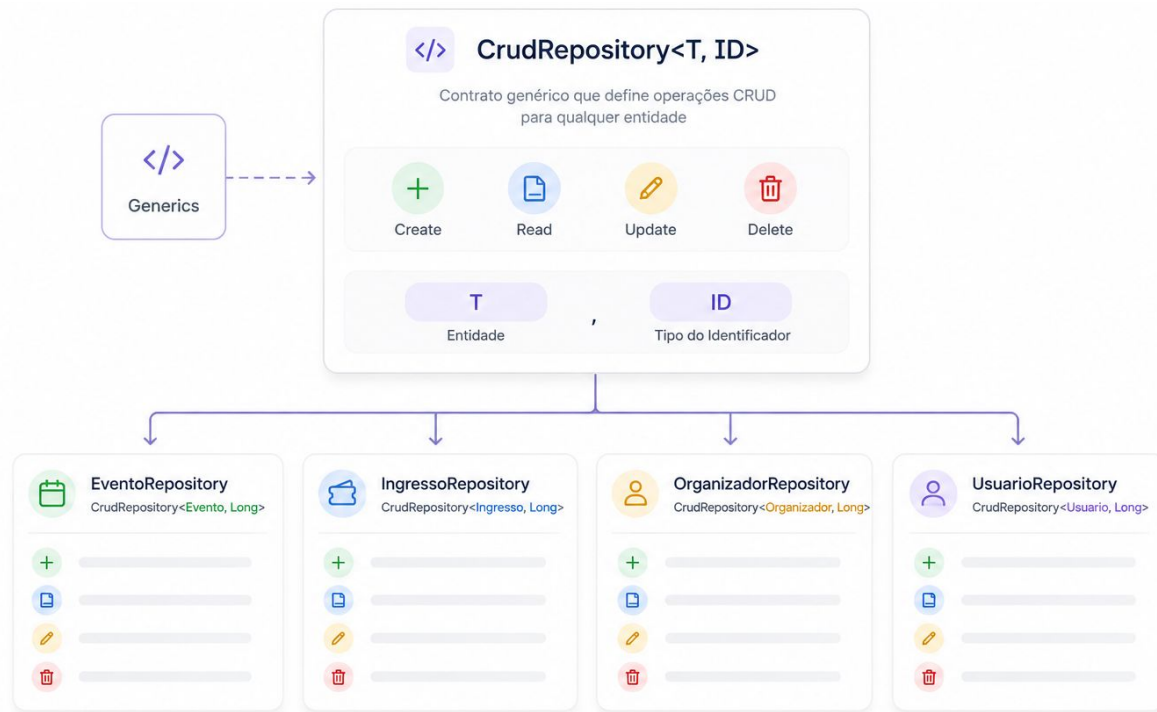
- ConfigProperties centraliza propriedades do application.properties utilizando injeção automática via @Value.



```
1 public class ConfigProperties {
2     @Value(key = "datasource.url")
3     private String url;
4
5     @Value(key = "datasource.username")
6     private String username;
7
8     @Value(key = "datasource.password")
9     private String password;
10
11     @Value(key = "datasource.driver-class-name")
12     private String driverClassName;
13
14     @Value(key = "datasource.hikari.maximum-pool-size")
15     private int maximumPoolSize;
16
17     @Value(key = "datasource.hikari.minimum-idle")
18     private int minimumIdle;
19
20     @Value(key = "datasource.hikari.connection-timeout")
21     private long connectionTimeout;
22 }
```



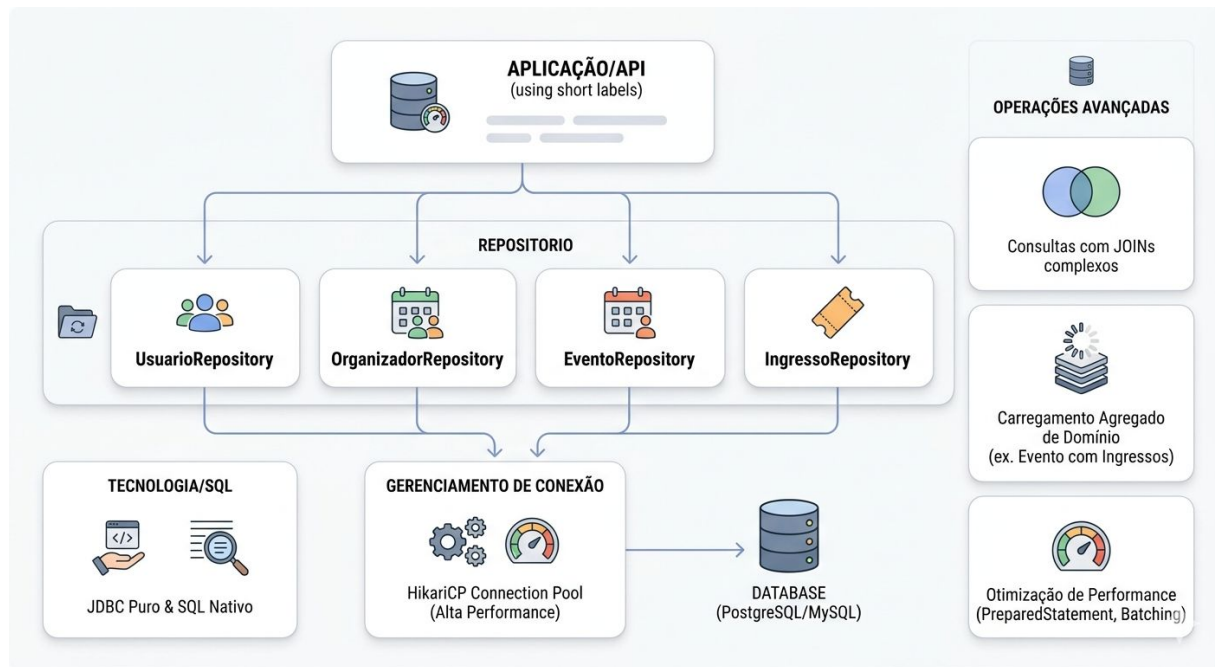
# Crud Repository e Generics



**CrudRepository<T, ID> padroniza operações CRUD utilizando Generics**

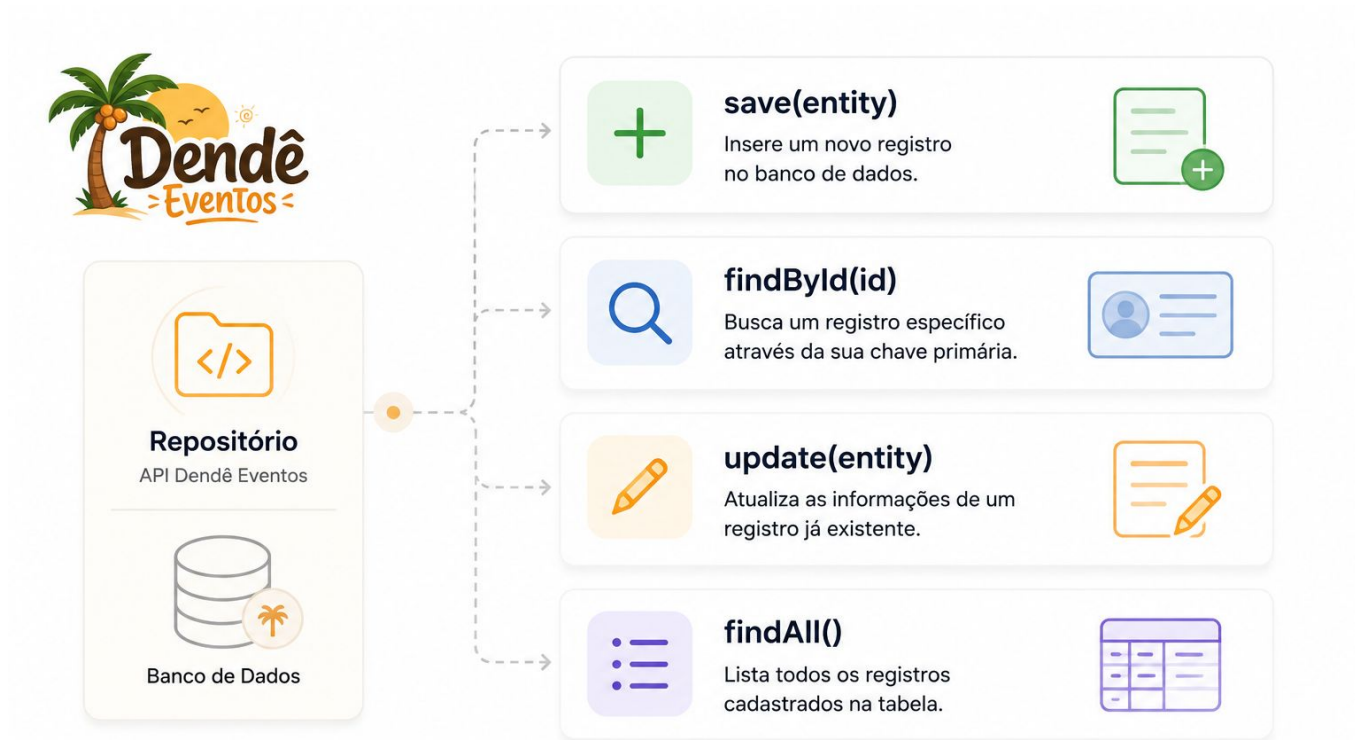
Dendê Eventos: Persistência e exceções

# Construção dos Repositories



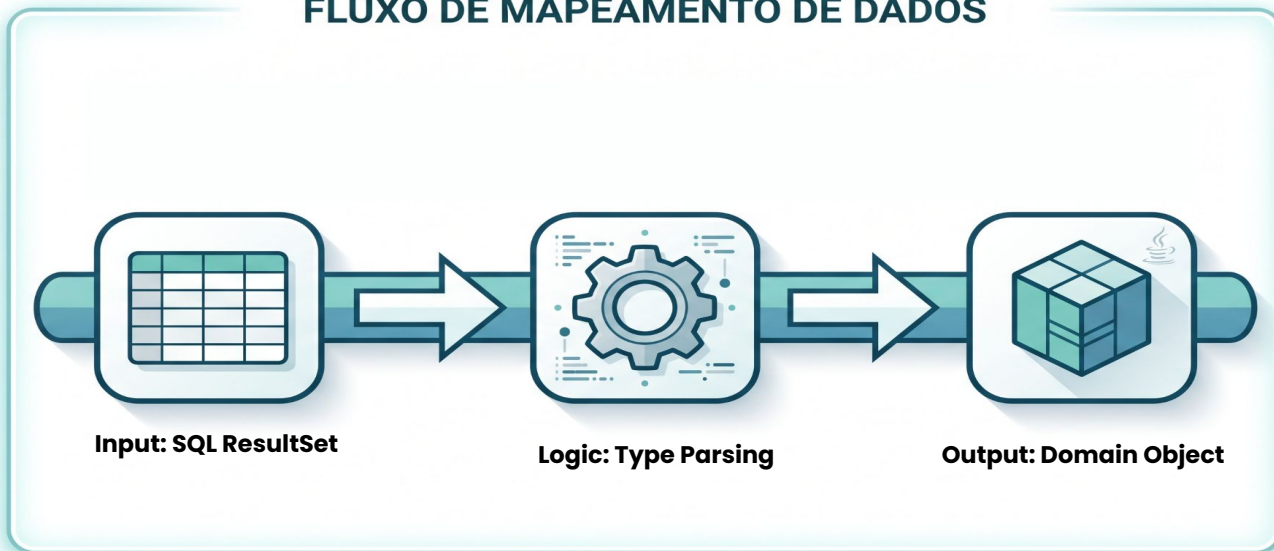
**Arquitetura da camada de persistência com JDBC Puro, HikariCP e divisão de repositórios do Dendê Eventos**

# Construção dos Repositories



# RowMapper

## FLUXO DE MAPEAMENTO DE DADOS



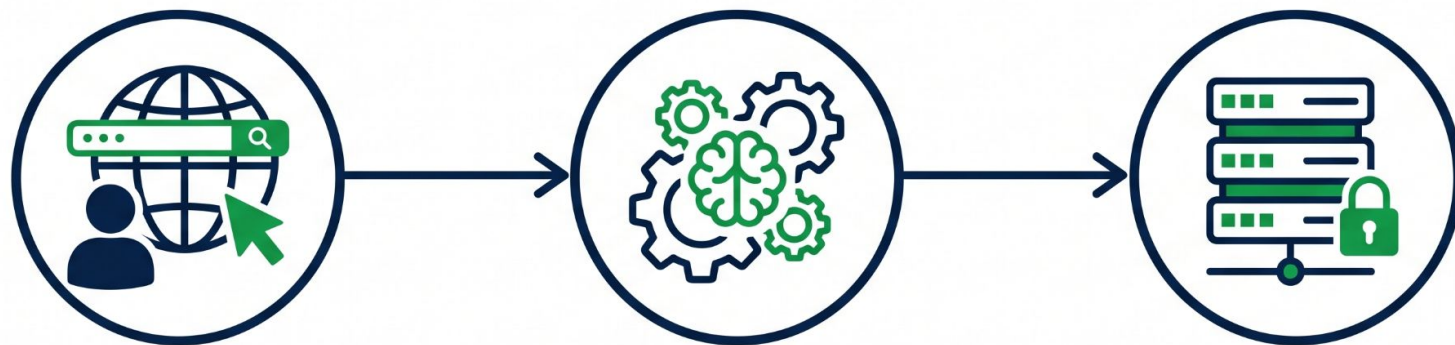
`mapRow(String[] row)`

# Tratamento de exceções

Exceções customizadas baseadas em RuntimeException centralizam o tratamento de erros de persistência, validação e regras de negócio, garantindo padronização e robustez da API.

| exception                                                                         |                                        |                                                                                     |
|-----------------------------------------------------------------------------------|----------------------------------------|-------------------------------------------------------------------------------------|
|  | CancelamentoNaoPermitidoException.java |  |
|  | CompralIngressoException.java          |  |
|  | EstornoNaoPermitidoException.java      |  |
|  | IngressoInvalidoException.java         |  |
|  | IngressoJaCanceladoException.java      |  |
|  | PessoaJaCadastradaException.java       |  |
|  | RecursoNaoEncontradoException.java     |  |

# Inversão de Controle: Arquitetura Spring



**RestController**  
(API Endpoint)

**Service Layer**  
(Business Logic)

**Repository**  
(Data Access)

@RestController → Recebe a Requisição

@Service → Regras de Negócio

@Repository → Acesso ao Banco de Dados MySQL

# Desserialização Automática com @RequestBody

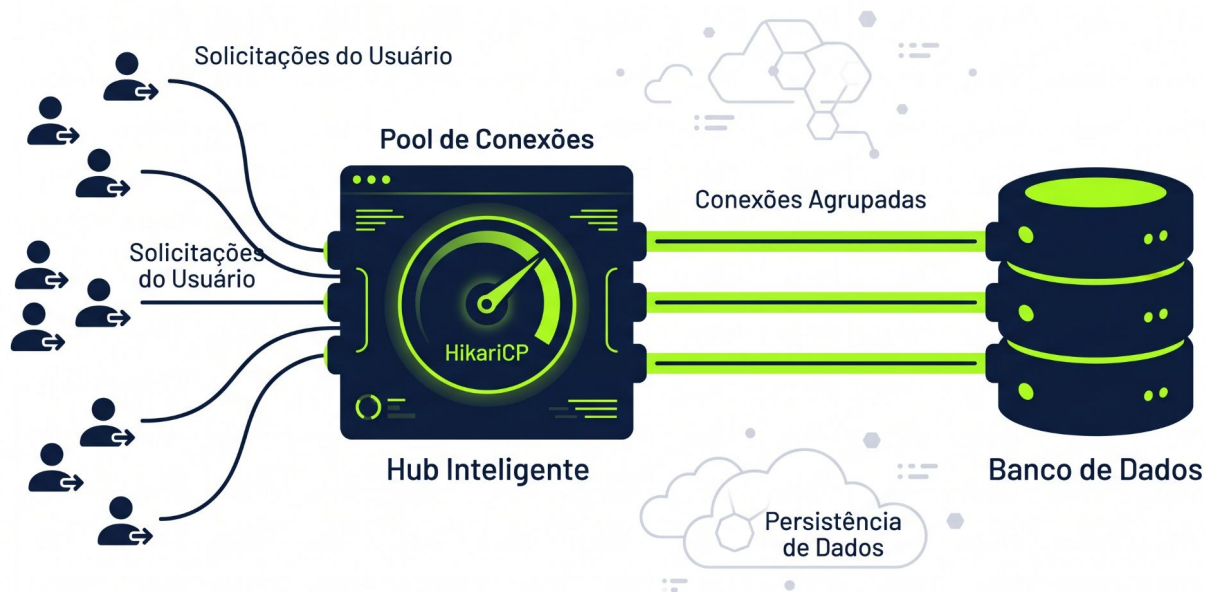


**Entrada:** Texto JSON (Postman)

**Motor:** Biblioteca Jackson

**Saída:** Objeto Java (`Usuario.java`)  
pronto para uso.

# Otimização de Memória: HikariCP & Hibernate



**HikariCP:** Pool de Conexões  
(Reciclagem e Eficiência)

**Hibernate:** Cache de 1º Nível  
(Menos idas ao banco)

**Resultado:** Prevenção de *Memory Leaks* e sobrecarga.



# Considerações finais

- Persistência com JDBC
- ConnectionPool com HikariCP
- ConfigProperties
- CrudRepository<T, ID>
- RowMapper<T>
- Repositories
- Exceções Customizadas

# Referências



ORACLE. *Java Documentation*. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 17 maio 2026.

ORACLE. *JDBC Documentation*. Disponível em: <https://docs.oracle.com/javase/tutorial/jdbc/>. Acesso em: 17 maio 2026.

MYSQL. *MySQL Documentation*. Disponível em: <https://dev.mysql.com/doc/>. Acesso em: 17 maio 2026.

BRETT WOOLDRIDGE. *HikariCP Documentation*. Disponível em: <https://github.com/brettwooldridge/HikariCP>. Acesso em: 17 maio 2026.

IBM. Creating sequence diagrams. IBM Docs, 2021. Disponível em: <https://www.ibm.com/docs/en/rational-soft-arch/10.0.0?topic=diagrams-creating-sequence>. Acesso em: 17 maio 2026.

# **Persistência, Exceções e Refatoração no Dendê Eventos**

**Autor:** André Almeida Gomes Neto  
Augusto Brandão Ferreira  
Deivid Souza dos Santos Oliveira  
Hellen Verena da Conceição Magalhães  
Ismael Rodrigues de Oliveira Neto  
João Vítor da Conceição de Almeida